LLM Architectures: Optimization, Gradient Descent, and Regularization

A Comprehensive Example-Driven Tutorial

Study Tutorial

April 6, 2026

Contents

1 Introduction **2**

2 Understanding Regressions **2**
- 2.1 What does "Regression" actually mean? . . . 2
- 2.2 Linear Regression: The Simplest Model . . . 3
- 2.3 Polynomial Regression: When Lines Are Not Enough . . . 3
- 2.4 Why "Logistic Regression" is called that . . . 4
- 2.5 Other Types of Regression (Quick Overview) . . . 5

3 Data Preprocessing: From Text to Numbers **6**
- 3.1 Step 1: Text Cleaning . . . 6
- 3.2 Step 2: Tokenization . . . 7
- 3.3 Step 3: Building a Vocabulary . . . 7
- 3.4 Step 4: Bag of Words (BoW) . . . 8

4 Building Logistic Regression in PyTorch **9**
- 4.1 What is a Tensor? . . . 9
- 4.2 Weight Initialization: Why It Matters . . . 9
- 4.3 Binary Cross-Entropy Loss: Grading the Model . . . 10
- 4.4 The Numerical Stability Problem . . . 11

5 Understanding Gradients by Example **12**

6 The Complete Training Loop **13**

7 Hyperparameter Experiments **15**
- 7.1 Learning Rate: Too Small, Just Right, Too Large . . . 15
- 7.2 Batch Size Effects . . . 15

8 Regularization: Preventing Overfitting **16**
- 8.1 L2 Penalty (Ridge) . . . 16
- 8.2 L1 Penalty (Lasso) . . . 17
- 8.3 Why L1 Creates Sparsity (Zero Weights) . . . 17
- 8.4 Proximal Descent: Getting True Zeros . . . 18

9 Advanced Optimization Algorithms **19**
9.1 Plain Gradient Descent 19
9.2 Momentum: Adding Inertia 20
9.3 AdaGrad: Per-Parameter Learning Rates 21
9.4 Adam: The Best of Both Worlds 22

10 Convex vs Non-Convex Optimization **23**
10.1 The Convex Bowl 23
10.2 The Six-Hump Camel Function 23
10.3 Key Observations from Experiments 23

11 Summary and Key Takeaways **24**

1 Introduction

Welcome to this comprehensive, step-by-step tutorial. Our single governing rule is: **all mathematics will be expressed as executable Python code**. No abstract equations, no Greek-letter formulas – just functions you can run, trace, and debug.

We will cover every major topic from the LLM Architectures assignment:

- What "Regression" actually means, the history behind the name, and how Linear, Polynomial, and Logistic Regression relate to each other.
- How raw text becomes numbers a computer can process (cleaning, tokenization, Bag-of-Words).
- Writing Logistic Regression from scratch in pure Python, then in PyTorch.
- Why numerical stability matters and how to prevent your training from crashing.
- How gradients work by example – what they are, how to compute them, and why they point "uphill."
- The full training loop: epochs, batches, shuffling, forward pass, backward pass, optimizer step.
- How learning rate and batch size affect convergence (with concrete experiments).
- L1 and L2 regularization: what they do, why L1 creates sparsity, and how to implement them.
- Four optimization algorithms implemented step-by-step: plain Gradient Descent, Momentum, AdaGrad, and Adam.
- Convex vs. non-convex optimization: the Bowl function and the Six-Hump Camel function.
- Proximal descent and why SGD cannot produce true zeros with L1.

2 Understanding Regressions

2.1 What does "Regression" actually mean?

The word "regression" comes from the 1800s. Francis Galton studied the heights of parents and children. He noticed that extremely tall parents tended to have children who were shorter than themselves – closer to the population average. He called this "regression toward mediocrity" (later renamed "regression to the mean").

The statistical tools Galton built to study these relationships became known as "regression analysis." Today, in machine learning, **regression** refers to predicting a continuous number (as opposed to **classification**, which predicts a category).

2.2 Linear Regression: The Simplest Model

Imagine you want to predict the price of a house based on its size. You believe bigger houses cost more, in a roughly proportional way.

```python
def linear_regression(square_footage):
    """
    The simplest possible model.
    We have ONE weight and ONE bias.
    The prediction is a straight line.
    """
    weight = 150.0   # dollars per square foot
    bias = 50000.0    # base price (land, location, etc.)

    predicted_price = (square_footage * weight) + bias
    return predicted_price

# Let's test it on a few houses:
print(linear_regression(500))    # Output: 125000.0
print(linear_regression(1000))   # Output: 200000.0
print(linear_regression(2000))   # Output: 350000.0
print(linear_regression(5000))   # Output: 800000.0

# Notice: the relationship is perfectly straight.
# Every additional square foot adds exactly $150.
```

Linear Regression: Predicting House Prices

Figure 1: Linear regression produces a perfectly straight line.

2.3 Polynomial Regression: When Lines Are Not Enough

What if luxury mansions sell at a premium? The relationship might curve upward for larger houses. We can capture this by adding a squared term:

```python
def polynomial_regression(square_footage):
    """
    We add a squared term so the price curves upward
```

```
    for very large houses.
    """
    weight_linear = 100.0     # base dollars per sqft
    weight_squared = 0.02     # premium scaling factor
    bias = 50000.0

    predicted_price = (
        (square_footage * weight_linear) +
        ((square_footage ** 2) * weight_squared) +
        bias
    )
    return predicted_price

# The curve accelerates for bigger houses:
print(polynomial_regression(500))    # Output: 105000.0
print(polynomial_regression(1000))   # Output: 170000.0
print(polynomial_regression(2000))   # Output: 330000.0
print(polynomial_regression(5000))   # Output: 1050000.0  <- premium!
```

Polynomial Regression: Curved Pricing

2.4 Why "Logistic Regression" is called that

Now consider a completely different problem: classifying emails as spam (1) or not-spam (0). If we use linear regression, the model might predict -4.5 or 12.3 but these numbers are meaningless as probabilities.

The solution? Scientists took the linear regression output and passed it through a special "squashing" function called the **logistic function** (also called the sigmoid). This function takes any number – no matter how large or negative – and squashes it into the range (0, 1).

Because they were still using the regression framework (weights times features plus bias), just with a squashing step bolted onto the end, the name **Logistic Regression** stuck permanently – even though it performs classification, not regression.

```
import math

def sigmoid(raw_number):
    """
    The logistic/sigmoid function.
    Input: any real number from -infinity to +infinity
    Output: a number strictly between 0 and 1

    How it works:
    1. Negate the input
    2. Raise e (2.718...) to that power
    3. Add 1
    4. Take the reciprocal (1 divided by the result)
    """
    step_1 = -raw_number            # negate
    step_2 = math.exp(step_1)       # e^(-x)
    step_3 = 1.0 + step_2           # 1 + e^(-x)
    step_4 = 1.0 / step_3           # 1 / (1 + e^(-x))
    return step_4

# Let's trace through some examples:
print(sigmoid(-10.0))   # 0.0000454  (very close to 0)
```

```python
print(sigmoid(-2.0))    # 0.1192     (low probability)
print(sigmoid(0.0))     # 0.5        (exactly in the middle)
print(sigmoid(2.0))     # 0.8808     (high probability)
print(sigmoid(10.0))    # 0.9999546  (very close to 1)
```

The Sigmoid Function Step by Step

```python
def logistic_regression_predict(features, weights, bias):
    """
    Full logistic regression prediction in three steps:

    Step 1: Compute the linear combination (same as linear regression)
    Step 2: Pass it through sigmoid
    Step 3: Threshold at 0.5 to get a class label
    """
    # Step 1: Linear combination
    raw_score = 0.0
    for feature_value, weight_value in zip(features, weights):
        raw_score += feature_value * weight_value
    raw_score += bias

    # Step 2: Squash to probability
    probability = sigmoid(raw_score)

    # Step 3: Convert probability to class
    if probability >= 0.5:
        predicted_class = 1  # spam / positive
    else:
        predicted_class = 0  # not spam / negative

    return predicted_class, probability

# Example: email with 3 features
# [number_of_exclamation_marks, contains_word_free, email_length]
features = [5, 1, 200]
weights = [0.3, 2.0, -0.01]
bias = -1.0

label, prob = logistic_regression_predict(features, weights, bias)
print(f"Probability of spam: {prob:.4f}")
print(f"Predicted class: {label}")
```

Complete Logistic Regression from Scratch

2.5 Other Types of Regression (Quick Overview)

For completeness, here are a few other regression types you may encounter:

```python
# 1. Ridge Regression = Linear Regression + L2 penalty on weights
#    (we cover L2 penalty in detail in the Regularization section)

# 2. Lasso Regression = Linear Regression + L1 penalty on weights
#    (forces some weights to zero -- feature selection!)

# 3. Softmax Regression = Logistic Regression extended to 3+ classes
#    Instead of one sigmoid, we use a "softmax" function that
#    distributes probability across multiple classes:
def softmax(scores):
```

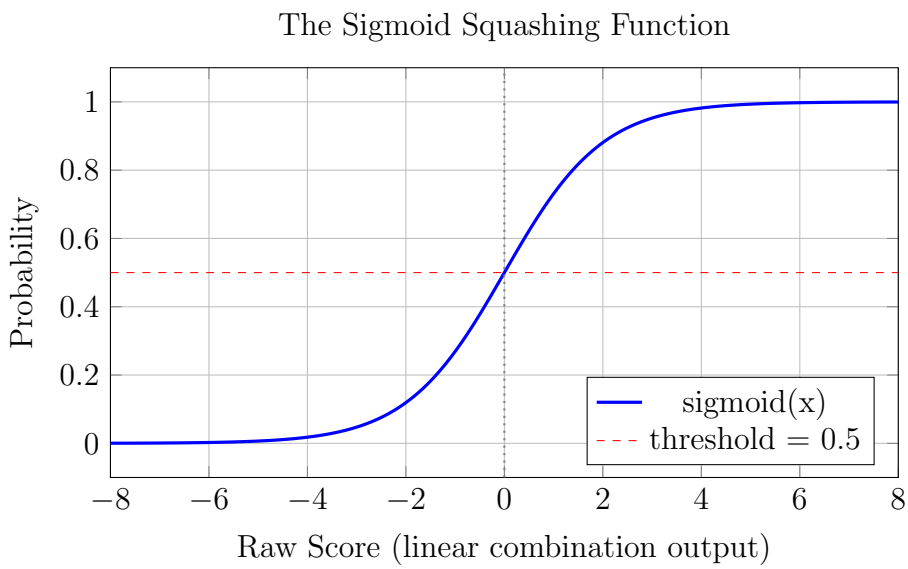

Figure 2: The sigmoid function maps any real number to a probability between 0 and 1. Values above the 0.5 threshold are classified as positive.

```
    """Convert a list of raw scores into probabilities that sum to 1.
    """
    max_score = max(scores)  # for numerical stability
    exp_scores = [math.exp(s - max_score) for s in scores]
    total = sum(exp_scores)
    return [e / total for e in exp_scores]

print(softmax([2.0, 1.0, 0.1]))
# Output: [0.659, 0.242, 0.099]  (sums to 1.0)
```

Overview of Regression Types

3 Data Preprocessing: From Text to Numbers

Machine learning models only understand numbers. Before training, we must convert every sentence into a list of numbers. This section walks through the complete pipeline.

3.1 Step 1: Text Cleaning

Raw text is messy. "Movie" and "movie" would be treated as separate words. "well-written" might not match "well written." We clean the text to make it consistent:

```
import re

def clean_text(text):
    """
    Normalize text so that minor variations don't create
    different features.

    Steps:
    1. Lowercase everything
    2. Replace hyphens with spaces ("well-written" -> "well written")
```

```
    3. Remove anything that isn't a letter, number, space, or basic punctuation
    4. Collapse multiple spaces into one
    """
    # Step 1
    text = text.lower()

    # Step 2
    text = text.replace('-', ' ')

    # Step 3: keep only [a-z, 0-9, spaces, period, comma, !, ?]
    text = re.sub(r'[^a-z0-9\s.,!?]', '', text)

    # Step 4: collapse whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Examples:
print(clean_text("I LOVED this Movie!!!"))
# Output: "i loved this movie!!!"

print(clean_text("Well-written & AMAZING film"))
# Output: "well written  amazing film"
```

Text Cleaning Function

3.2 Step 2: Tokenization

Tokenization means splitting the cleaned text into individual units (tokens). In our simple case, we just split on spaces:

```
def tokenize(text):
    """Split text into individual words."""
    return text.split()

print(tokenize("i loved this movie"))
# Output: ['i', 'loved', 'this', 'movie']

# Note: Modern LLMs use "sub-word" tokenizers (like BPE)
# that can split "unbelievable" into ["un", "believ", "able"]
# so they never encounter unknown words.
# For now, word-level tokenization is sufficient.
```

Simple Tokenizer

3.3 Step 3: Building a Vocabulary

We need a fixed-size dictionary mapping words to integer indices. We only keep the most frequent words (e.g., top 10,000) because our model needs fixed-length input vectors.

```
from collections import Counter

def build_vocabulary(list_of_texts, top_k=10000):
    """
    Count every word across all texts.
    Keep only the top_k most frequent ones.
```

```python
    Return a dictionary: word -> index
    """
    word_counter = Counter()

    for text in list_of_texts:
        words = tokenize(text)
        word_counter.update(words)

    # most_common returns [(word, count), ...] sorted by frequency
    most_common_words = word_counter.most_common(top_k)

    # Assign an index to each word
    vocab = {}
    for index, (word, count) in enumerate(most_common_words):
        vocab[word] = index

    return vocab

# Example with tiny data:
texts = [
    "i loved this movie",
    "this movie was terrible",
    "movie was great i loved it",
]
vocab = build_vocabulary(texts, top_k=100)
print(vocab)
# {'movie': 0, 'i': 1, 'loved': 2, 'this': 3, 'was': 4,
#  'terrible': 5, 'great': 6, 'it': 7}
```

Building a Vocabulary from Training Data

3.4 Step 4: Bag of Words (BoW)

Now we convert each sentence into a vector of word counts. The vector has one slot for every word in the vocabulary.

```python
def text_to_bow_vector(text, vocab):
    """
    Convert a sentence into a fixed-length vector.
    Each position corresponds to a word in the vocabulary.
    The value at that position is how many times the word appears.
    """
    vector = [0] * len(vocab)  # start with all zeros
    words = tokenize(text)

    for word in words:
        if word in vocab:
            index = vocab[word]
            vector[index] += 1
        # Words not in vocab are silently ignored

    return vector

# Using our vocab from above:
vec1 = text_to_bow_vector("i loved this movie", vocab)
print(f"'i loved this movie' -> {vec1}")
# Output: [1, 1, 1, 1, 0, 0, 0, 0]
#          movie=1, i=1, loved=1, this=1, rest=0
```

```python

vec2 = text_to_bow_vector("movie movie movie", vocab)
print(f"'movie movie movie' -> {vec2}")
# Output: [3, 0, 0, 0, 0, 0, 0, 0]
#  The word "movie" appears 3 times

vec3 = text_to_bow_vector("unknown words here", vocab)
print(f"'unknown words here' -> {vec3}")
# Output: [0, 0, 0, 0, 0, 0, 0, 0]
#  None of these words are in our vocabulary!
```

Bag of Words Conversion

4 Building Logistic Regression in PyTorch

Now we move from pure Python to PyTorch. PyTorch gives us two critical superpowers: (1) it can process thousands of examples simultaneously using matrix multiplication, and (2) it automatically computes gradients for us.

4.1 What is a Tensor?

A tensor is just a multi-dimensional array, like a list of lists, but optimized for speed. Think of it as a supercharged NumPy array that also tracks how it was computed so it can calculate gradients.

```python
import torch

# A 1D tensor (a vector):
v = torch.tensor([1.0, 2.0, 3.0])
print(v)        # tensor([1., 2., 3.])
print(v.shape)  # torch.Size([3])

# A 2D tensor (a matrix) -- 3 rows, 2 columns:
m = torch.tensor([[1.0, 2.0],
                  [3.0, 4.0],
                  [5.0, 6.0]])
print(m.shape)  # torch.Size([3, 2])

# Matrix multiplication: (3x2) @ (2x1) = (3x1)
weights = torch.tensor([[0.5], [-0.3]])
result = torch.matmul(m, weights)
print(result)
# tensor([[-0.1000],  <- 1*0.5 + 2*(-0.3) = -0.1
#         [ 0.3000],  <- 3*0.5 + 4*(-0.3) = 0.3
#         [ 0.7000]]) <- 5*0.5 + 6*(-0.3) = 0.7
```

Tensors 101

4.2 Weight Initialization: Why It Matters

Before training starts, we must choose initial values for the weights. This choice matters more than you might think.

```python
import torch
import torch.nn as nn
```

```python

class LogisticRegression(nn.Module):
    def __init__(self, n_features, init="zeros"):
        super().__init__()

        if init == "zeros":
            # All weights start at 0.
            # Safe for convex problems like logistic regression.
            # But BAD for deep neural networks (symmetry problem).
            w = torch.zeros(n_features, 1)

        elif init == "random":
            # Small random values (scale ~0.01).
            # Breaks symmetry. Each weight starts slightly different.
            # We use SMALL values to avoid exploding gradients.
            w = torch.randn(n_features, 1) * 0.01

        elif isinstance(init, torch.Tensor):
            # Use a specific tensor (useful for experiments)
            w = init.clone().detach().view(n_features, 1)
        else:
            raise ValueError("init must be 'zeros', 'random', or a Tensor")

        # nn.Parameter tells PyTorch: "track gradients for this"
        self.w = nn.Parameter(w)
        self.b = nn.Parameter(torch.zeros(1))

    def forward(self, x):
        """
        The forward pass:
        1. Multiply inputs by weights, add bias  (linear regression)
        2. Squash through sigmoid                 (logistic part)
        """
        logits = torch.matmul(x, self.w) + self.b  # shape: (N, 1)
        probabilities = torch.sigmoid(logits)        # shape: (N, 1)
        return probabilities

    def predict(self, x):
        """Convert probabilities to class labels (0 or 1)."""
        probs = self.forward(x)
        return (probs >= 0.5).float()
```

Weight Initialization Strategies

4.3 Binary Cross-Entropy Loss: Grading the Model

The loss function measures how wrong the model is. For binary classification, we use Binary Cross-Entropy (BCE), also called log-loss. The core idea: **penalize the model heavily when it is confident but wrong**.

```python
import math

def log_loss_single_example(true_label, predicted_prob):
    """
    Compute loss for ONE example.

```

```
    Think of it this way:
    - If true_label is 1, we want predicted_prob close to 1.
      Loss = -log(predicted_prob)
      If predicted_prob = 0.99, loss = -log(0.99) = 0.01  (tiny!)
      If predicted_prob = 0.01, loss = -log(0.01) = 4.6   (huge!)

    - If true_label is 0, we want predicted_prob close to 0.
      Loss = -log(1 - predicted_prob)
      If predicted_prob = 0.01, loss = -log(0.99) = 0.01  (tiny!)
      If predicted_prob = 0.99, loss = -log(0.01) = 4.6   (huge!)
    """
    if true_label == 1:
        return -math.log(predicted_prob)
    else:
        return -math.log(1 - predicted_prob)

# Let's trace exact values:
print("=== TRUE LABEL IS 1 (positive) ===")
print(f"Pred=0.99 -> Loss={log_loss_single_example(1, 0.99):.4f}")
print(f"Pred=0.90 -> Loss={log_loss_single_example(1, 0.90):.4f}")
print(f"Pred=0.50 -> Loss={log_loss_single_example(1, 0.50):.4f}")
print(f"Pred=0.10 -> Loss={log_loss_single_example(1, 0.10):.4f}")
print(f"Pred=0.01 -> Loss={log_loss_single_example(1, 0.01):.4f}")
```

Understanding Log Loss Through Examples

4.4 The Numerical Stability Problem

There is a catastrophic bug lurking in the code above. Watch what happens if the model predicts exactly 0.0:

```
# This CRASHES the program:
# log_loss_single_example(1, 0.0)
# --> math.log(0.0) --> -infinity --> ERROR!

# The fix: "clamp" predictions so they never hit exactly 0 or 1
def stable_bce_loss(true_label, predicted_prob):
    """
    The SAFE version of log loss.
    We force predictions into the range [epsilon, 1-epsilon]
    so log() never receives 0.
    """
    epsilon = 1e-15  # 0.000000000000001

    # Clamp: if pred < epsilon, set it to epsilon
    #        if pred > 1-epsilon, set it to 1-epsilon
    safe_pred = max(epsilon, min(1.0 - epsilon, predicted_prob))

    if true_label == 1:
        return -math.log(safe_pred)
    else:
        return -math.log(1.0 - safe_pred)

# Now it works safely:
print(stable_bce_loss(1, 0.0))       # ~34.5 (large but finite!)
print(stable_bce_loss(0, 1.0))       # ~34.5 (large but finite!)
```

```
print(stable_bce_loss(1, 0.999999))  # ~0.000001 (tiny)
```

The Crash

```
import torch

def binary_cross_entropy_loss(y_pred, y_true):
    """
    Compute average BCE loss over a batch of examples.

    y_pred: tensor of predicted probabilities, shape (N, 1)
    y_true: tensor of true labels (0 or 1), shape (N, 1)
    """
    epsilon = 1e-15

    # Clamp predictions for safety
    y_pred = torch.clamp(y_pred, epsilon, 1.0 - epsilon)

    # Compute loss for each example
    losses = -(
        y_true * torch.log(y_pred) +
        (1 - y_true) * torch.log(1 - y_pred)
    )

    # Return the average loss across all examples
    return losses.mean()
```

PyTorch Version of BCE Loss

5 Understanding Gradients by Example

Before diving into optimizers, let's make sure gradients are crystal clear. A gradient tells you: "if I increase this weight by a tiny amount, how much does the loss change?"

```
def compute_gradient_manually(f, x, tiny_step=0.0001):
    """
    Approximate the gradient of function f at point x.

    The gradient is the slope: how much does f(x) change
    when we nudge x by a tiny amount?

    gradient = (f(x + tiny_step) - f(x - tiny_step)) / (2 * tiny_step)
    """
    value_right = f(x + tiny_step)
    value_left = f(x - tiny_step)
    gradient = (value_right - value_left) / (2 * tiny_step)
    return gradient

# Example: f(x) = x^2
def f_squared(x):
    return x * x

# At x=3, the gradient should be 2*3 = 6
print(compute_gradient_manually(f_squared, 3.0))
# Output: 6.00000 (correct!)

# At x=-2, the gradient should be 2*(-2) = -4
```

```python
print(compute_gradient_manually(f_squared, -2.0))
# Output: -4.00000 (correct!)

# At x=0, the gradient should be 0 (bottom of the bowl)
print(compute_gradient_manually(f_squared, 0.0))
# Output: 0.00000 (correct! -- this is the minimum!)
```

Gradients: The Manual Way

```python
import torch

# PyTorch computes gradients automatically!
x = torch.tensor(3.0, requires_grad=True)

# Forward pass: compute the function
y = x * x  # y = x^2

# Backward pass: compute dy/dx
y.backward()

# The gradient is stored in x.grad
print(x.grad)  # tensor(6.)  -- same answer, zero manual work!
```

Gradients: The PyTorch Way (Autograd)

The key insight: **gradients point uphill**. If the gradient is positive, the function increases as x increases. To minimize the function, we move in the *opposite* direction (subtract the gradient).

6 The Complete Training Loop

Training a model means repeating these steps thousands of times until the loss is small. Let's trace through every single step.

```python
def train_logistic_regression(
    X_train, y_train, X_val, y_val,
    learning_rate=0.01, epochs=20, batch_size=100,
    init="zeros"
):
    """
    A complete training loop for logistic regression.

    TERMINOLOGY:
    - Epoch: one complete pass through the entire training set
    - Batch: a small chunk of the training set (e.g., 100 examples)
    - Iteration: one weight update (processing one batch)
    """
    # Convert data to PyTorch tensors
    X_train_t = torch.tensor(X_train, dtype=torch.float32)
    y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
    X_val_t = torch.tensor(X_val, dtype=torch.float32)
    y_val_t = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)

    n_samples = X_train_t.shape[0]
    n_features = X_train_t.shape[1]

    # Step 1: Create the model
```

```python
    model = LogisticRegression(n_features, init=init)

    # Step 2: Create the optimizer
    # This object will handle the "subtract gradient * lr" logic
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

    # Step 3: Loop over epochs
    for epoch in range(epochs):

        # Shuffle the data at the start of each epoch.
        # WHY? So the model doesn't memorize the order of examples.
        permutation = torch.randperm(n_samples)
        X_shuffled = X_train_t[permutation]
        y_shuffled = y_train_t[permutation]

        # Step 4: Loop over mini-batches
        for start in range(0, n_samples, batch_size):
            end = start + batch_size
            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            # Step 4a: FORWARD PASS
            # (model computes predictions)
            predictions = model(X_batch)

            # Step 4b: COMPUTE LOSS
            loss = binary_cross_entropy_loss(predictions, y_batch)

            # Step 4c: BACKWARD PASS
            # (PyTorch computes gradients automatically)
            loss.backward()

            # Step 4d: OPTIMIZER STEP
            # (Update weights using gradients)
            optimizer.step()

            # Step 4e: CLEAR GRADIENTS
            # (Reset for next batch; otherwise they accumulate!)
            optimizer.zero_grad()

        # Step 5: Evaluate after each epoch
        with torch.no_grad():  # don't track gradients during eval
            train_pred = model(X_train_t)
            train_loss = binary_cross_entropy_loss(train_pred, y_train_t)
            train_acc = ((model.predict(X_train_t) == y_train_t)
                         .float().mean().item())

            val_pred = model(X_val_t)
            val_loss = binary_cross_entropy_loss(val_pred, y_val_t)
            val_acc = ((model.predict(X_val_t) == y_val_t)
                       .float().mean().item())

        print(f"Epoch {epoch+1}/{epochs} | "
              f"Train Loss: {train_loss:.4f} | "
              f"Train Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} | "
```

```
                f"Val Acc: {val_acc:.4f}")
```

The Full Training Loop Explained

7 Hyperparameter Experiments

Different learning rates and batch sizes produce very different training behaviors. Let's understand why.

7.1 Learning Rate: Too Small, Just Right, Too Large

```
def demonstrate_learning_rates():
    """
    Imagine the loss landscape is a simple bowl: loss = weight^2
    The gradient at any point is: 2 * weight
    We start at weight = 5.0 and try to reach weight = 0.0
    """
    def gradient_at(weight):
        return 2 * weight

    # --- Learning rate too SMALL (0.01) ---
    weight = 5.0
    print("=== lr=0.01 (too small) ===")
    for step in range(10):
        grad = gradient_at(weight)
        weight = weight - 0.01 * grad
        print(f"  Step {step+1}: weight={weight:.4f}")
    # After 10 steps: weight is still ~4.1. Barely moved!

    # --- Learning rate just RIGHT (0.3) ---
    weight = 5.0
    print("\n=== lr=0.3 (good) ===")
    for step in range(10):
        grad = gradient_at(weight)
        weight = weight - 0.3 * grad
        print(f"  Step {step+1}: weight={weight:.4f}")
    # After 10 steps: weight is ~0.005. Almost perfect!

    # --- Learning rate too LARGE (0.6) ---
    weight = 5.0
    print("\n=== lr=0.6 (too large -- DIVERGES!) ===")
    for step in range(10):
        grad = gradient_at(weight)
        weight = weight - 0.6 * grad
        print(f"  Step {step+1}: weight={weight:.4f}")
    # After 10 steps: weight oscillates wildly: 5, -1, 0.2, ...
```

Learning Rate Intuition

7.2 Batch Size Effects

```
# SMALL BATCH (e.g., 10 examples):
# - Each gradient estimate is NOISY because 10 examples
#   might not represent the full dataset well
# - Updates happen FREQUENTLY (many batches per epoch)
```

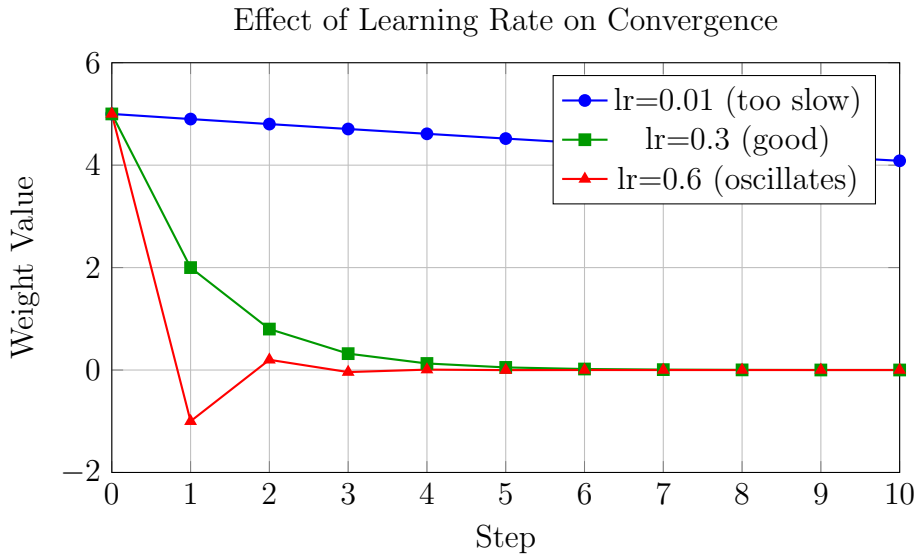

Figure 3: With lr=0.01 the model crawls. With lr=0.3 it converges smoothly. With lr=0.6 the weights bounce back and forth (though they happen to converge here because the overshooting shrinks).

```
# - Can escape local minima due to noise
# - Can be unstable

# LARGE BATCH (e.g., 1000 examples):
# - Each gradient estimate is ACCURATE
# - Updates happen RARELY (few batches per epoch)
# - Smooth, stable convergence
# - Uses more memory
# - Can get stuck in sharp local minima

# RULE OF THUMB: Start with batch_size=64 or 128
# and adjust based on your GPU memory and convergence behavior.
```

Batch Size Intuition

8 Regularization: Preventing Overfitting

Overfitting happens when a model memorizes the training data instead of learning general patterns. A model with 10,000 weights and only 1,000 training examples has plenty of room to "cheat" – it can assign specific weights to match individual examples rather than learning useful patterns.

Regularization adds a **penalty** to the loss function that punishes large weights. This forces the model to use simpler, more generalizable patterns.

8.1 L2 Penalty (Ridge)

```
def compute_l2_penalty(weights, lambda_strength):
    """
    L2 penalty = lambda * sum(w_i^2) for all weights.

    Properties:
    - Large weights get taxed HEAVILY (because squaring amplifies them)
```

```
    - Small weights get taxed BARELY (0.01^2 = 0.0001, almost nothing)
    - Result: all weights shrink, but none become exactly zero
    """
    penalty = 0.0
    for w in weights:
        penalty += w * w
    return lambda_strength * penalty

# Example:
weights = [3.0, -2.0, 0.5, 0.01, -0.001]
print(f"L2 penalty: {compute_l2_penalty(weights, 0.01):.6f}")
# = 0.01 * (9 + 4 + 0.25 + 0.0001 + 0.000001) = 0.01 * 13.2501 = 0.1325
```

L2 Penalty: Tax on Squared Weights

8.2 L1 Penalty (Lasso)

```
def compute_l1_penalty(weights, lambda_strength):
    """
    L1 penalty = lambda * sum(|w_i|) for all weights.

    Properties:
    - Large weights get taxed proportionally
    - Small weights ALSO get taxed proportionally (no mercy!)
    - Result: unimportant weights get pushed all the way to ZERO
    - This = automatic feature selection!
    """
    penalty = 0.0
    for w in weights:
        penalty += abs(w)
    return lambda_strength * penalty

# Same example:
weights = [3.0, -2.0, 0.5, 0.01, -0.001]
print(f"L1 penalty: {compute_l1_penalty(weights, 0.01):.6f}")
# = 0.01 * (3 + 2 + 0.5 + 0.01 + 0.001) = 0.01 * 5.511 = 0.05511
```

L1 Penalty: Tax on Absolute Weights

8.3 Why L1 Creates Sparsity (Zero Weights)

The key difference is in how the penalty pushes weights during training. Let's trace through multiple training steps to see it in action:

```
def simulate_weight_decay(initial_weight, n_steps, lr, lam, mode):
    """
    Simulate what happens to a SINGLE weight over time
    when only the regularization penalty is acting
    (assume the data gradient is zero for simplicity).
    """
    w = initial_weight
    history = [w]

    for step in range(n_steps):
        if mode == "l2":
            # L2 gradient = 2 * lambda * w
            # The force toward zero WEAKENS as w gets smaller
            gradient = 2 * lam * w
        elif mode == "l1":
```

```
            # L1 gradient = lambda * sign(w)
            # The force toward zero is CONSTANT
            if w > 0:
                gradient = lam
            elif w < 0:
                gradient = -lam
            else:
                gradient = 0

        w = w - lr * gradient
        history.append(w)

    return history

# Trace a weight starting at 1.0 over 50 steps:
l2_history = simulate_weight_decay(1.0, 50, lr=0.1, lam=0.5, mode="l2")
l1_history = simulate_weight_decay(1.0, 50, lr=0.1, lam=0.5, mode="l1")

print("L2 final weight:", l2_history[-1])  # Small but NOT zero
print("L1 final weight:", l1_history[-1])  # Reaches zero and stays
```

Tracing L1 vs L2 Weight Decay Over Time

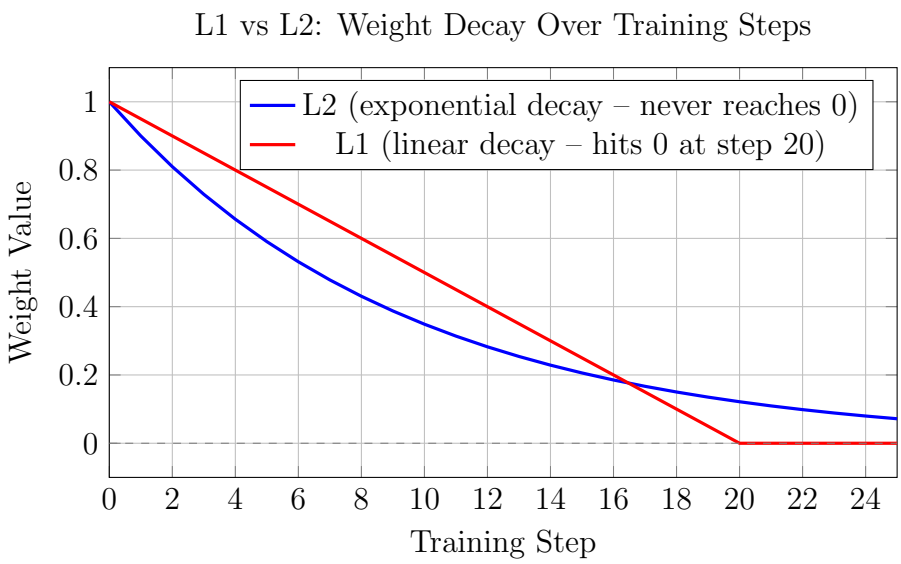

Figure 4: L2 regularization decays weights exponentially (they approach zero but never arrive). L1 regularization decays weights linearly at a constant rate, hitting exactly zero and staying there.

8.4 Proximal Descent: Getting True Zeros

Standard SGD with L1 cannot produce *exact* zeros because the constant push overshoots past zero and bounces back. Proximal descent fixes this with a two-step procedure:

```
def proximal_step(weight, data_gradient, lr, lam):
    """
    Two-step procedure that achieves TRUE zeros:

    Step 1: Normal gradient step (using only data loss)
```

```
    Step 2: "Soft thresholding" -- snap small weights to zero
    """
    # Step 1: gradient step (ignore regularization here)
    intermediate = weight - lr * data_gradient

    # Step 2: soft thresholding
    threshold = lr * lam

    if intermediate > threshold:
        # Weight is large enough to survive; just shrink it
        return intermediate - threshold
    elif intermediate < -threshold:
        # Weight is large negative; shrink toward zero
        return intermediate + threshold
    else:
        # Weight is close to zero -- SNAP IT TO ZERO
        return 0.0

# Example:
# A weight at 0.03 with threshold 0.05 gets snapped to exactly 0
print(proximal_step(0.03, 0.0, lr=0.1, lam=0.5))  # Output: 0.0
# A weight at 0.10 with threshold 0.05 survives as 0.05
print(proximal_step(0.10, 0.0, lr=0.1, lam=0.5))  # Output: 0.05
```

Proximal Descent for True Sparsity

9 Advanced Optimization Algorithms

Plain SGD has a critical limitation: it treats every gradient equally and has no memory of past steps. This section implements four optimizers from scratch so you can see exactly how each one works.

9.1 Plain Gradient Descent

```
def gradient_descent(f, start_x, start_y, lr=0.01, n_steps=100):
    """
    Optimize a 2D function f(x, y) using vanilla gradient descent.
    Returns the trajectory (list of (x, y) points visited).
    """
    import torch

    x = torch.tensor(start_x, dtype=torch.float32, requires_grad=True)
    y = torch.tensor(start_y, dtype=torch.float32, requires_grad=True)

    trajectory = [(x.item(), y.item())]

    for step in range(n_steps):
        # Compute the function value
        loss = f(x, y)

        # Compute gradients (how does loss change w.r.t. x and y?)
        loss.backward()

        # Update: move opposite to the gradient
        with torch.no_grad():
```

```
            x -= lr * x.grad
            y -= lr * y.grad

        # Clear gradients for next step
        x.grad.zero_()
        y.grad.zero_()

        trajectory.append((x.item(), y.item()))

    return trajectory
```

Gradient Descent: The Simplest Optimizer

9.2 Momentum: Adding Inertia

SGD only considers the current gradient. If the landscape has a flat region, SGD stalls completely. Momentum fixes this by maintaining a "velocity" – like a ball rolling downhill that builds up speed.

```
def momentum_optimizer(f, start_x, start_y, lr=0.01, beta=0.9, n_steps
    =100):
    """
    beta = friction coefficient (0.9 means keep 90% of velocity)

    The update rule each step:
    1. velocity = beta * old_velocity + lr * current_gradient
    2. position = position - velocity

    This means: even if the current gradient is tiny,
    the accumulated velocity keeps us moving!
    """
    import torch

    x = torch.tensor(start_x, dtype=torch.float32, requires_grad=True)
    y = torch.tensor(start_y, dtype=torch.float32, requires_grad=True)

    # Initialize velocity to zero
    vel_x = 0.0
    vel_y = 0.0

    trajectory = [(x.item(), y.item())]

    for step in range(n_steps):
        loss = f(x, y)
        loss.backward()

        with torch.no_grad():
            # Update velocity (keeps 90% of old + adds new gradient)
            vel_x = beta * vel_x + lr * x.grad.item()
            vel_y = beta * vel_y + lr * y.grad.item()

            # Move using velocity (not raw gradient!)
            x -= vel_x
            y -= vel_y

        x.grad.zero_()
        y.grad.zero_()
        trajectory.append((x.item(), y.item()))

```

```
    return trajectory
```

Momentum: The Boulder Analogy

9.3 AdaGrad: Per-Parameter Learning Rates

Different parameters may need different learning rates. A weight for a rare but important word should take big steps; a weight for a common word like "the" should take tiny steps.

```
def adagrad_optimizer(f, start_x, start_y, lr=0.5, n_steps=100):
    """
    AdaGrad maintains a running sum of squared gradients for each
    parameter. Parameters that have received large gradients in the
    past get smaller learning rates.

    Problem: the learning rate only ever DECREASES.
    After many steps, it can become so small that learning stops.
    """
    import torch

    x = torch.tensor(start_x, dtype=torch.float32, requires_grad=True)
    y = torch.tensor(start_y, dtype=torch.float32, requires_grad=True)

    # Running sum of squared gradients
    sum_sq_x = 0.0
    sum_sq_y = 0.0
    eps = 1e-8  # to avoid division by zero

    trajectory = [(x.item(), y.item())]

    for step in range(n_steps):
        loss = f(x, y)
        loss.backward()

        with torch.no_grad():
            gx = x.grad.item()
            gy = y.grad.item()

            # Accumulate squared gradients
            sum_sq_x += gx * gx
            sum_sq_y += gy * gy

            # Effective lr = base_lr / sqrt(accumulated_squares)
            x -= lr * gx / (sum_sq_x ** 0.5 + eps)
            y -= lr * gy / (sum_sq_y ** 0.5 + eps)

        x.grad.zero_()
        y.grad.zero_()
        trajectory.append((x.item(), y.item()))

    return trajectory
```

AdaGrad: Adaptive Learning Rates

9.4 Adam: The Best of Both Worlds

Adam combines Momentum (first moment / mean of gradients) with AdaGrad-style scaling (second moment / variance of gradients), plus a bias correction trick.

```python
def adam_optimizer(f, start_x, start_y, lr=0.01, n_steps=100):
    """
    Adam maintains TWO running averages per parameter:
    - m: exponential moving average of gradients (like momentum)
    - v: exponential moving average of squared gradients (like AdaGrad)

    The bias correction fixes the fact that m and v start at 0.
    Without it, the first few steps would be misleadingly small.
    """
    import torch, math

    x = torch.tensor(start_x, dtype=torch.float32, requires_grad=True)
    y = torch.tensor(start_y, dtype=torch.float32, requires_grad=True)

    beta1 = 0.9    # momentum decay rate
    beta2 = 0.999  # variance decay rate
    eps = 1e-8

    # Running averages (start at zero)
    m_x, m_y = 0.0, 0.0  # first moment (mean)
    v_x, v_y = 0.0, 0.0  # second moment (variance)

    trajectory = [(x.item(), y.item())]

    for step in range(1, n_steps + 1):
        loss = f(x, y)
        loss.backward()

        with torch.no_grad():
            gx = x.grad.item()
            gy = y.grad.item()

            # Update first moment (momentum-style)
            m_x = beta1 * m_x + (1 - beta1) * gx
            m_y = beta1 * m_y + (1 - beta1) * gy

            # Update second moment (variance-style)
            v_x = beta2 * v_x + (1 - beta2) * gx * gx
            v_y = beta2 * v_y + (1 - beta2) * gy * gy

            # Bias correction
            m_x_hat = m_x / (1 - beta1 ** step)
            m_y_hat = m_y / (1 - beta1 ** step)
            v_x_hat = v_x / (1 - beta2 ** step)
            v_y_hat = v_y / (1 - beta2 ** step)

            # Update parameters
            x -= lr * m_x_hat / (math.sqrt(v_x_hat) + eps)
            y -= lr * m_y_hat / (math.sqrt(v_y_hat) + eps)

        x.grad.zero_()
        y.grad.zero_()
        trajectory.append((x.item(), y.item()))
```

```

    return trajectory
```

Adam: The Modern Default Optimizer

10 Convex vs Non-Convex Optimization

10.1 The Convex Bowl

A convex function has exactly one minimum. No matter where you start, if you follow the gradient downhill, you will always reach the same optimal point.

```
def bowl(x, y):
    """
    f(x, y) = x^2 + 4*y^2

    This is an elliptical bowl centered at (0, 0).
    The y-direction is steeper (coefficient 4 vs 1).
    The unique global minimum is at (0, 0) with value 0.
    """
    return x**2 + 4 * y**2
```

Convex Bowl Function

10.2 The Six-Hump Camel Function

This is a non-convex function with multiple local minima. Optimizers can get trapped!

```
def camel(x, y):
    """
    A nasty non-convex function with:
    - 2 global minima at (0.0898, -0.7126) and (-0.0898, 0.7126)
      with value approximately -1.0316
    - Several LOCAL minima (traps for optimizers!)
    - Valleys, ridges, and saddle points
    """
    return (
        (4 - 2.1 * x**2 + x**4 / 3) * x**2 +
        x * y +
        (-4 + 4 * y**2) * y**2
    )
```

Six-Hump Camel Function

10.3 Key Observations from Experiments

```
# On the CONVEX BOWL:
# - All four optimizers converge to (0, 0). No surprises.
# - Adam and AdaGrad adapt per-axis learning rates, which helps
#   because the bowl is steeper in the y-direction.
# - Momentum overshoots slightly but then settles down.

# On the CAMEL function:
# - Plain GD often gets stuck in a LOCAL minimum.
# - Momentum sometimes escapes local minima (its inertia carries
```

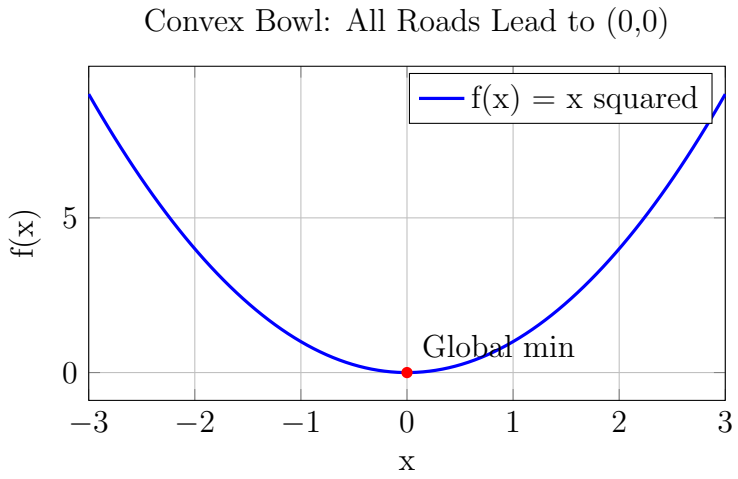

Figure 5: A convex function. No matter where you start on this curve, following the slope downhill always brings you to the same bottom point.

```
#   it over small bumps) but can also overshoot the global minimum.
# - AdaGrad slows down too much and gets stuck.
# - Adam is the most robust but can STILL get trapped depending
#   on the starting point.
#
# CRITICAL INSIGHT: No optimizer guarantees finding the global
# minimum of a non-convex function. The starting point matters!
```

Comparing Optimizers – Key Takeaways

11 Summary and Key Takeaways

1. **Regression types**: Linear regression predicts continuous values. Logistic regression adds a sigmoid to predict probabilities. The name "logistic regression" is historical – it actually does classification.

2. **Data pipeline**: Text goes through cleaning, tokenization, vocabulary building, and Bag-of-Words conversion to become numbers.

3. **Numerical stability**: Always clamp predictions away from exactly 0 and 1 before taking logarithms, or your training will crash.

4. **Gradients**: They tell you the slope. To minimize a function, move opposite to the gradient (subtract it).

5. **Learning rate**: Too small = slow. Too large = unstable. Experiment to find the right value.

6. **Batch size**: Small = noisy but fast updates. Large = smooth but memory-heavy. Typical range: 32–256.

7. **L1 vs L2**: L2 shrinks all weights proportionally (never reaches zero). L1 applies constant force toward zero, creating sparsity (automatic feature selection).

8. **Optimizers**: Momentum adds inertia to power through flat regions. AdaGrad adapts learning rates per parameter. Adam combines both and is the modern default.

9. **Convex vs non-convex**: On convex functions, all optimizers find the global minimum. On non-convex functions, optimizers can get trapped in local minima – the starting point matters.